Design Pattern Request Batch

Intento e definizione

L'intento del pattern Request Batch è combinare diverse richieste individuali in un'unica richiesta aggregata per ottimizzare l'utilizzo della rete e migliorare le prestazioni complessive del sistema.

Il **problema** è che, in un sistema distribuito, inviare numerose richieste di piccole dimensioni è inefficiente. Ogni singola richiesta comporta:

→ **latenza di rete**: il tempo necessario per il viaggio di andata e ritorno (round-trip) dei pacchetti.

→ **overhead di elaborazione**: il server deve gestire l'apertura e la chiusura delle connessioni e, soprattutto, i processi di serializzazione e deserializzazione dei dati per ogni singola chiamata.

→ esempio: se un servizio deve leggere dati da un database 10.000 volte, effettuare 10.000 connessioni separate causerà un'alta latenza e un throughput limitato. Raggruppare queste richieste in blocchi, ad esempio da 100, riduce drasticamente il tempo totale e il numero di round-trip.

La **soluzione** proposta è accumulare le richieste e inviarle come un singolo "lotto" (batch).

Funzionamento

Il funzionamento del Request Batch prevede i seguenti passaggi:

1. **Accumulo**: il client (o un proxy) non invia subito la richiesta, ma la accumula. L'invio del batch avviene quando si raggiunge un certo numero massimo di richieste o dopo un intervallo di tempo predefinito.
2. **Identificazione**: poiché le richieste sono raggruppate, è fondamentale associare a ciascuna di esse un ID univoco per poter successivamente mappare la risposta corretta alla richiesta originale.
3. **Invio e ricezione**: il batch viaggia come un'unica richiesta di rete. Il server ricevente (Receiver) disaggrega il batch, processa le singole operazioni e restituisce le risposte aggregate in un unico batch di ritorno.
4. **Smistamento**: il client riceve il batch di risposte e le distribuisce ai richiedenti originali in base ai relativi ID univoci.

Componenti strutturali

Il pattern identifica quattro componenti logici principali:

- **Request Sender**: raccoglie le richieste individuali dai client.
- **Batch Assembler**: aggrega le richieste raccolte in un unico oggetto batch.
- **Batch Processor** (Receiver): lato server, riceve il batch, lo elabora e produce le risposte corrispondenti.
- **Response Splitter**: lato client, riceve il batch di risposte e le distribuisce ai client che avevano originato le richieste.

Conseguenze

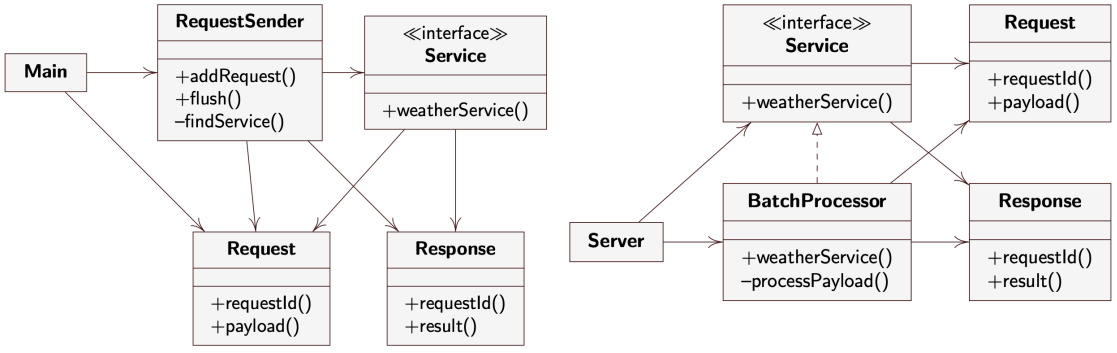
L'utilizzo del Request Batch comporta:

Benefici

- riduzione significativa del numero di messaggi e dei round-trip di rete.
- miglioramento del throughput e dell'utilizzo delle risorse.
- diminuzione dell'overhead dovuto alla serializzazione e alla gestione delle connessioni.

Compromessi

- **latenza di accumulo**: le prime richieste del batch devono aspettare che il batch sia pieno o che scada il timeout prima di essere inviate, introducendo un ritardo intenzionale.
- **complessità**: maggiore complessità nella gestione della corrispondenza tra richieste e risposte (smistamento).
- **gestione dei guasti**: se l'elaborazione dell'intero batch fallisce a causa di una singola richiesta malformata, l'intero blocco potrebbe essere compromesso.

Esempio applicativo

Un servizio fornisce le previsioni meteo per diverse città. Invece di fare una chiamata remota per ogni città, il client utilizza un RequestSender che accumula le richieste. Quando inviate, il server riceve una lista di oggetti Request e restituisce una lista di oggetti Response in un'unica interazione.